

Labsheet-10

Character/Digit Classification using CNN

```
In [1]: # Step 1: Import Libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# Step 2: Load MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Step 3: Preprocess images
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# Step 4: Build CNN model
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation="relu",
                           input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

# Step 5: Compile model
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Step 6: Train CNN
history = model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

# Step 7: Evaluate model
loss, accuracy = model.evaluate(x_test, y_test)
print("Test Accuracy:", round(accuracy * 100, 2), "%")

# Classification report
y_pred = np.argmax(model.predict(x_test), axis=1)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix
```

```

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# Step 8: Prediction on unseen test images
predictions = model.predict(x_test[:5])

plt.figure(figsize=(10, 2))
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.title("Pred: " + str(np.argmax(predictions[i])))
    plt.axis("off")
plt.show()

# Step 9: Plot training and validation accuracy
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.show()

# Step 10: Observation
print("\nObservation:")
print("CNN successfully classified handwritten digits from the MNIST dataset.")
print("The model learned image features automatically using convolution and pool

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— 4s 0us/step

```

c:\Users\vivek\AppData\Local\Programs\Python\Python312\Lib\site-packages\keras\s
c\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input_shape`
`/`input_dim` argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Epoch 1/5

844/844  **17s** 18ms/step - accuracy: 0.9517 - loss: 0.1609 - val_accuracy: 0.9843 - val_loss: 0.0540

Epoch 2/5

844/844  **15s** 17ms/step - accuracy: 0.9841 - loss: 0.0504 - val_accuracy: 0.9862 - val_loss: 0.0474

Epoch 3/5

844/844  **13s** 16ms/step - accuracy: 0.9892 - loss: 0.0338 - val_accuracy: 0.9900 - val_loss: 0.0347

Epoch 4/5

844/844  **13s** 15ms/step - accuracy: 0.9920 - loss: 0.0251 - val_accuracy: 0.9900 - val_loss: 0.0359

Epoch 5/5

844/844  **13s** 16ms/step - accuracy: 0.9933 - loss: 0.0196 - val_accuracy: 0.9885 - val_loss: 0.0429

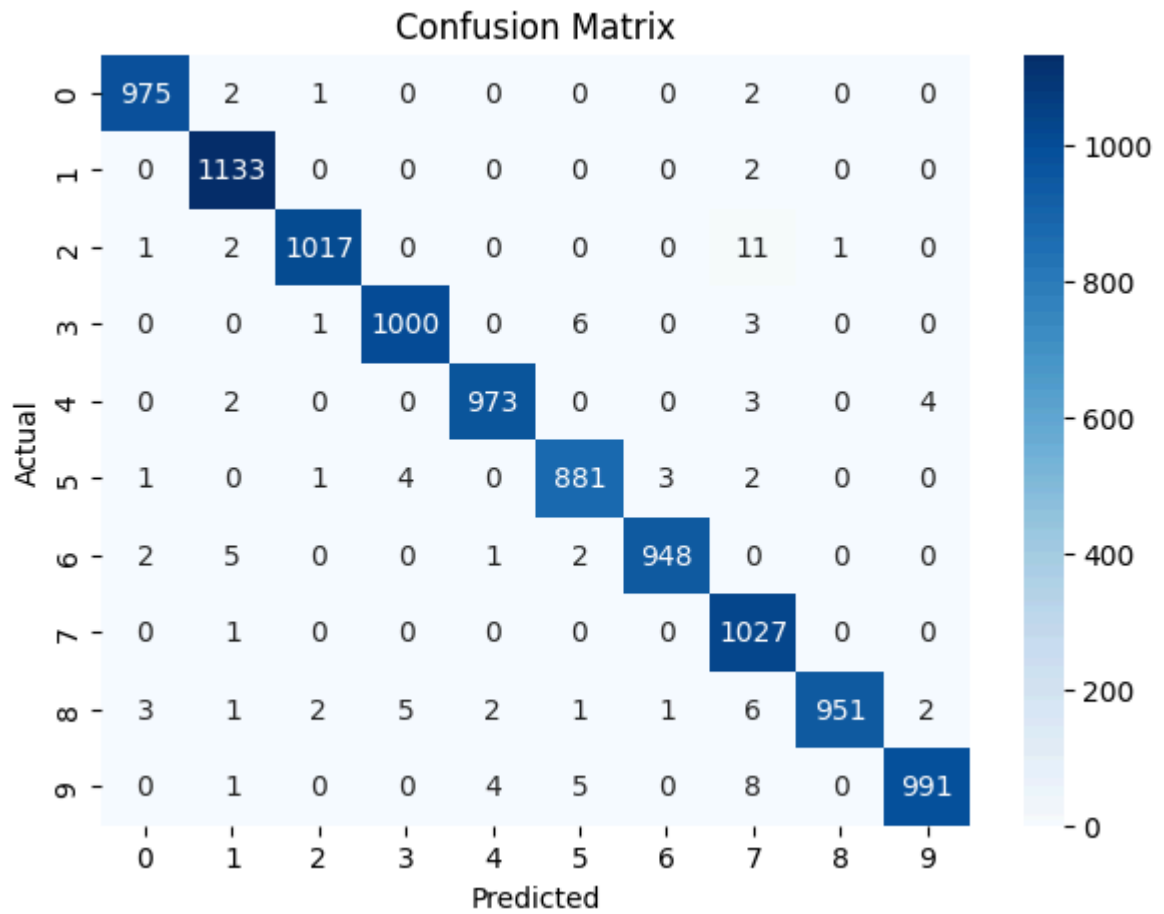
313/313  **1s** 4ms/step - accuracy: 0.9896 - loss: 0.0361

Test Accuracy: 98.96 %

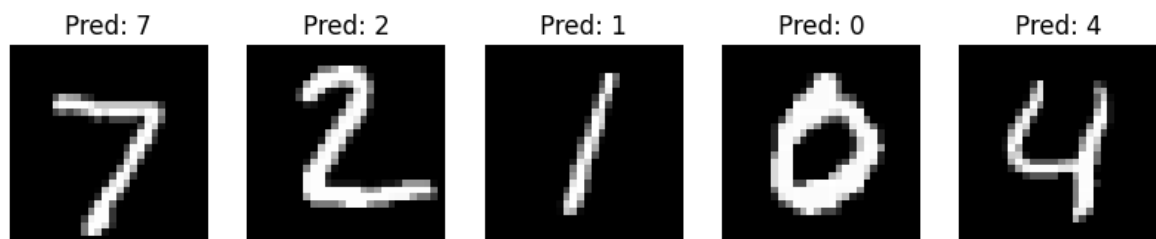
313/313  **1s** 4ms/step

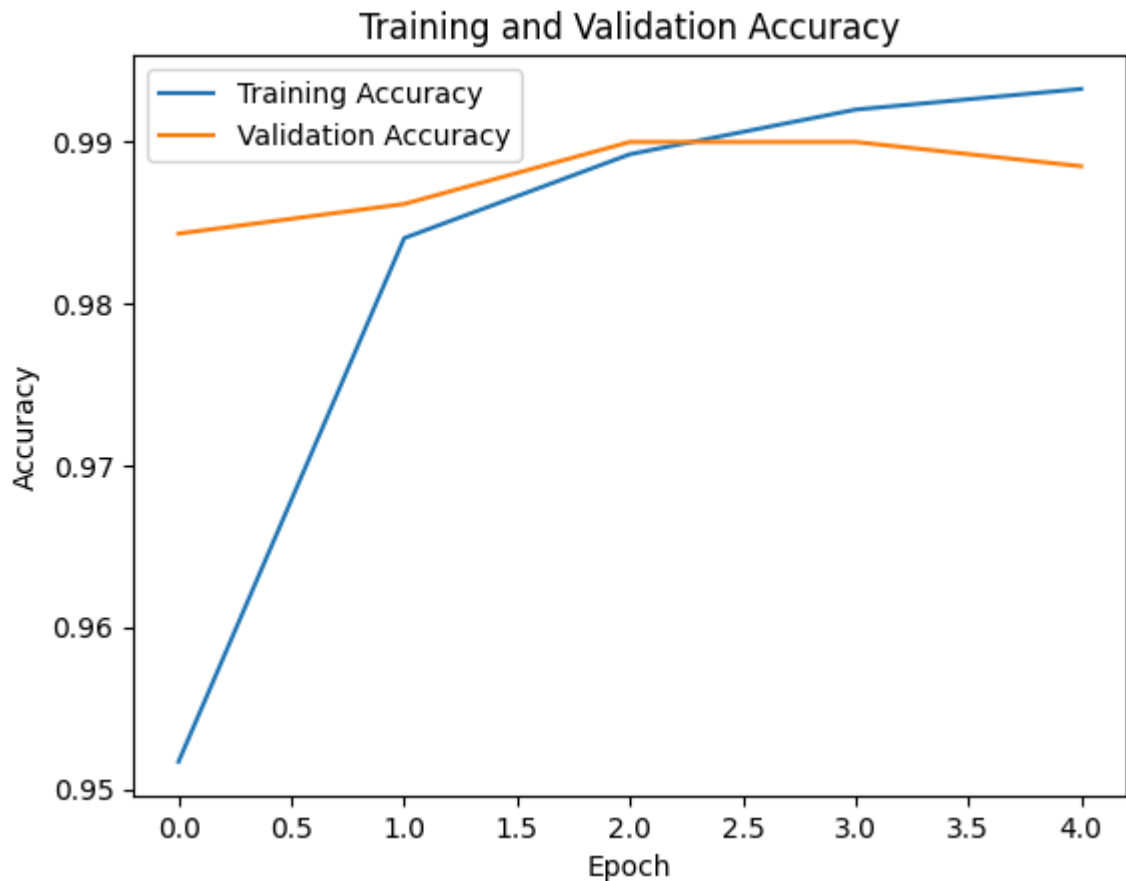
Classification Report:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	980
1	0.99	1.00	0.99	1135
2	1.00	0.99	0.99	1032
3	0.99	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.98	0.99	0.99	892
6	1.00	0.99	0.99	958
7	0.97	1.00	0.98	1028
8	1.00	0.98	0.99	974
9	0.99	0.98	0.99	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000



1/1 ————— 0s 51ms/step





Observation:

CNN successfully classified handwritten digits from the MNIST dataset.

The model learned image features automatically using convolution and pooling layers.

Questions and Answers

1. What is image classification? How does it differ from object detection and image segmentation?

Image Classification assigns a class or label to the entire image.

Object Detection identifies objects and locates them using bounding boxes.

Image Segmentation divides an image into regions and assigns a class to each pixel.

2. Explain the architecture and working principle of a Convolutional Neural Network (CNN).

A **CNN** is a deep learning model designed mainly for image processing. Its main layers are:

- **Convolutional Layer:** Extracts features such as edges, shapes, and textures.
- **Pooling Layer:** Reduces the spatial size of feature maps.
- **Fully Connected Layer:** Uses extracted features for classification.
- **Output Layer:** Produces the final class probabilities.

The image passes through these layers, features are automatically learned, and the network predicts the image class.

3. What is the role of Convolutional Layers, Pooling Layers, and Fully Connected Layers in a CNN?

- **Convolutional Layer:** Extracts important visual features from an image.
- **Pooling Layer:** Reduces image dimensions and computational complexity while retaining important features.
- **Fully Connected Layer:** Combines the extracted features and performs classification.

4. Why is image normalization performed before training a deep learning model?

Image normalization scales pixel values to a smaller range, commonly **0 to 1**. It helps the model train faster and more efficiently, improves numerical stability, and can lead to better convergence during training.

5. Explain the purpose of activation functions such as ReLU and Softmax in CNNs.

- **ReLU (Rectified Linear Unit):** Introduces non-linearity into the network and helps the model learn complex patterns. It is commonly used in hidden layers.
- **Softmax:** Converts the final layer outputs into probability values for different classes. It is commonly used for multi-class classification.

6. What is a Confusion Matrix? How is it used to evaluate classification performance?

A **Confusion Matrix** is a table that compares the actual class labels with the predicted class labels.

It contains values such as:

- **True Positive (TP):** Correctly predicted positive samples.
- **True Negative (TN):** Correctly predicted negative samples.
- **False Positive (FP):** Negative samples incorrectly predicted as positive.
- **False Negative (FN):** Positive samples incorrectly predicted as negative.

It helps identify which classes are being correctly or incorrectly classified.

7. Differentiate between Accuracy, Precision, Recall, and F1-Score with suitable examples.

- **Accuracy:** Measures the percentage of all predictions that are correct.

Formula:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

- **Precision:** Measures how many predicted positive samples are actually positive.

Formula:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

- **Recall:** Measures how many actual positive samples are correctly detected.

Formula:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

- **F1-Score:** Represents the harmonic mean of Precision and Recall.

Formula:

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

For example, in disease-image classification, **Recall** is important when missing a positive case can be significant.

8. Compare traditional feature-based image classification methods (e.g., SIFT/HOG + SVM) with CNN-based classification.

Feature	SIFT/HOG + SVM	CNN
Feature Extraction	Handcrafted	Automatically learned
Classification	SVM or other classifier	Neural network
Training Data	Usually less	Usually more
Feature Learning	Manual	Automatic
Complexity	Lower for small tasks	Higher
Performance	Good for specific tasks	Strong for complex image tasks

CNNs can learn hierarchical features directly from images, reducing the need for manual feature extraction.

9. Mention five real-world applications of CNN-based image classification.

1. **Handwritten Digit Recognition**
2. **Face Recognition**
3. **Medical Image Classification**
4. **Object and Product Recognition**
5. **Plant Disease Detection**

10. How can techniques such as Data Augmentation, Transfer Learning, and Hyperparameter Tuning improve the performance of deep learning-based image classification models?

- **Data Augmentation:** Creates variations of training images using operations such as rotation, flipping, cropping, and scaling. It helps improve generalization.
- **Transfer Learning:** Uses a model pre-trained on a large dataset and adapts it to a new classification task, reducing training requirements.
- **Hyperparameter Tuning:** Finds suitable values for parameters such as learning rate, batch size, number of layers, and epochs to improve model performance.

Together, these techniques can help improve **accuracy, generalization, training efficiency, and robustness** of CNN-based image classification models.